

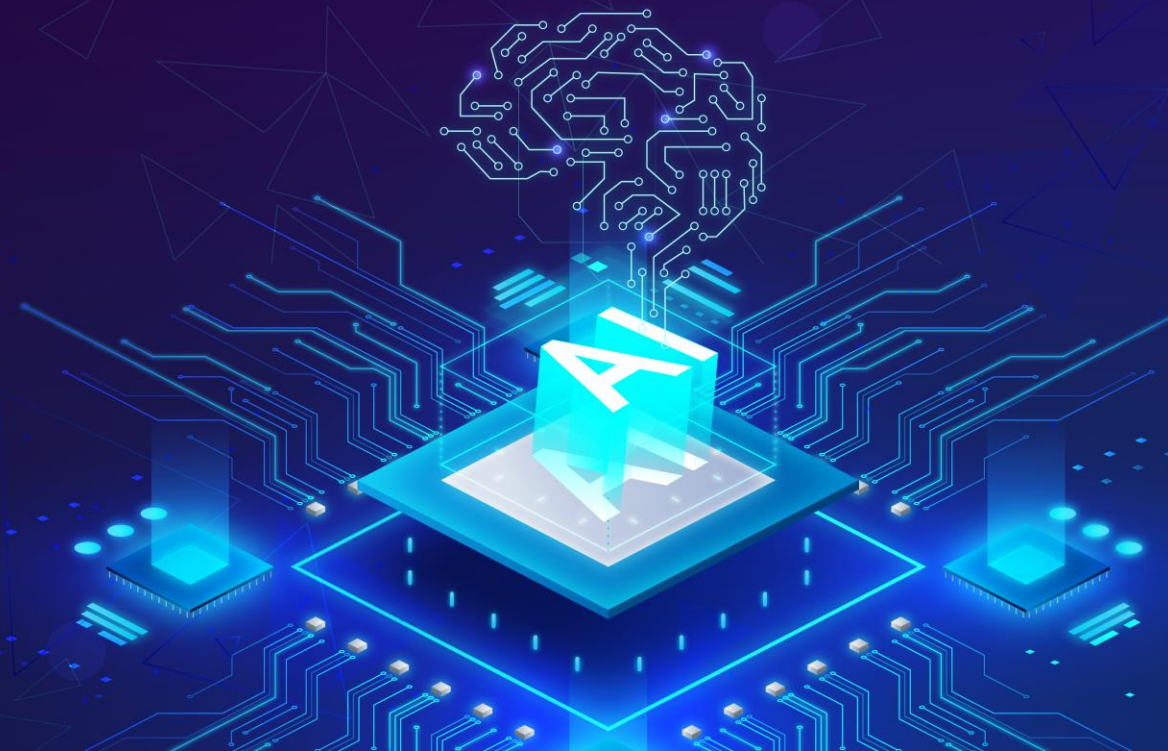


ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

Nhóm chuyên môn Trí tuệ nhân tạo

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Ví dụ về tìm kiếm có đối thủ



1. Trò chơi Tic-tac-toe

2. Trò chơi Cờ vua

Bài học này cung cấp cho người học:

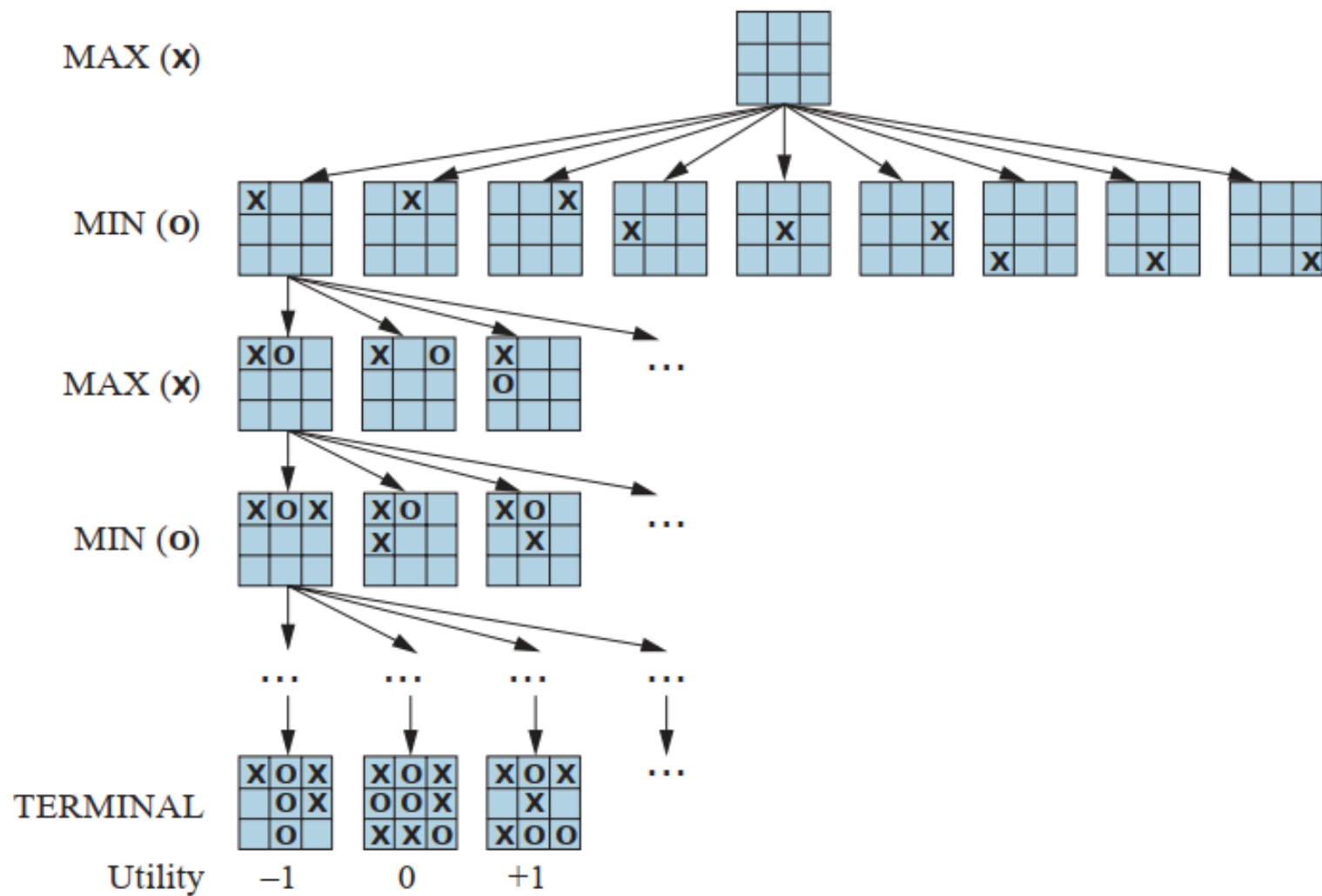
1. Ví dụ về xây dựng trò chơi **Tic-tac-toe** với Minimax và Alpha-beta (tìm kiếm tối ưu)
2. Xây dựng trò chơi **Cờ vua** với Minimax và Alpha-beta (tìm kiếm ước lượng)

1. Trò chơi Tic-tac-toe

2. Trò chơi Cờ vua

1. TRÒ CHƠI TIC-TAC-TOE

- Biểu diễn trạng thái (state)



1. TRÒ CHƠI TIC-TAC-TOE

- Biểu diễn trạng thái (state)

```
def initBoard():  
    return [[None] * Size for i in range(Size)]
```

```
(x) plays (0,0):  
x   .   .  
   .   o   .  
   .   .   .  
[[True, None, None], [None, False, None], [None, None, None]]
```

1. TRÒ CHƠI TIC-TAC-TOE

- Xây dựng và cấu hình thuật giải Minimax

```
function MINIMAX-DECISION(state) returns an action
```

```
   $v \leftarrow \text{MAX-VALUE}(\textit{state})$ 
```

```
  return the action in SUCCESSORS(state) with value  $v$ 
```

```
function MAX-VALUE(state) returns a utility value
```

```
  if TERMINAL-TEST(state) then return UTILITY(state)
```

```
   $v \leftarrow -\infty$ 
```

```
  for  $a, s$  in SUCCESSORS(state) do
```

```
     $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
```

```
  return  $v$ 
```

```
function MIN-VALUE(state) returns a utility value
```

```
  if TERMINAL-TEST(state) then return UTILITY(state)
```

```
   $v \leftarrow \infty$ 
```

```
  for  $a, s$  in SUCCESSORS(state) do
```

```
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
```

```
  return  $v$ 
```

1. TRÒ CHƠI TIC-TAC-TOE

- Xây dựng hàm tiện ích

```
def testLine(line):  
    if line == [True]*Size:           #X wins  
        return 1  
    if line == [False]*Size:         #O wins  
        return -1  
    if True in line and False in line: #draw line  
        return 0  
    return None                     #not yet concluded
```


1. TRÒ CHƠI TIC-TAC-TOE



■ Xây dựng hàm tiện ích

```
def utility(board):
    allLines = [row for row in board]
    allLines.extend([[row[i] for row in board] for i in range(Size)])
    allLines.append([board[i][i] for i in range(Size)])
    allLines.append([board[i][Size-i-1] for i in range(Size)])

    drawFlag = True
    for line in allLines:
        test = testLine(line)
        if test == 1 or test == -1:
            return test
        if test == None:
            drawFlag = False

    if drawFlag: return 0
    return None
```

#there's un-drawn line

1. TRÒ CHƠI TIC-TAC-TOE



- Xây dựng và cấu hình thuật giải Minimax

```
def maxValue(board):  
    utilityValue = utility(board)  
    if utilityValue != None: return (utilityValue, 0, 0)  
  
    valMax = -2  
    for (row, col) in possibleMoves(board):  
        newBoard = playXO(board, True, row, col)  
        val = minValue(newBoard)[0]  
  
        if valMax < val:  
            (valMax, xMax, yMax) = (val, row, col)  
  
    return (valMax, xMax, yMax)
```

1. TRÒ CHƠI TIC-TAC-TOE



■ Xây dựng và cấu hình thuật giải Minimax

You play: 1 1

```
. . .  
. x .  
. . .
```

Machine plays: 0 0

Numbers of moves checked: 53200

```
o . .  
. x .  
. . .
```

You play: 1 0

```
o . .  
x x .  
. . .
```

Machine plays: 1 2

Numbers of moves checked: 994

```
o . .  
x x o  
. . .
```

You play: 1 1

```
. . .  
. x .  
. . .
```

Machine plays: 0 0

Numbers of moves checked: 3131

```
o . .  
. x .  
. . .
```

You play: 1 0

```
o . .  
x x .  
. . .
```

Machine plays: 1 2

Numbers of moves checked: 210

```
o . .  
x x o  
. . .
```

NỘI DUNG TIẾP THEO



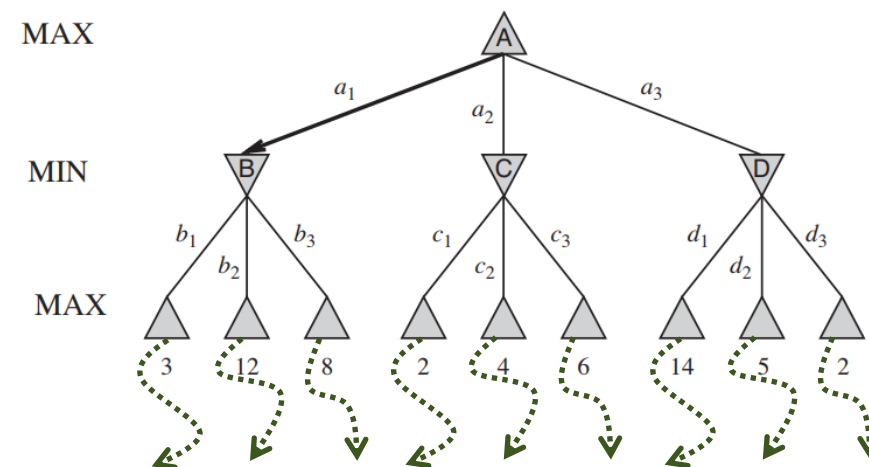
1. Trò chơi Tic-tac-toe

2. Trò chơi Cờ vua

2. TRÒ CHƠI CỜ VUA

- Ngừng tìm kiếm sớm

- Ngừng tìm kiếm (**cutoff**) tại một độ sâu hay điều kiện nào đó
- Chiến lược tìm kiếm **tối ưu** → **không tối ưu**
- Sử dụng hàm đánh giá trạng thái (Evaluation)



Hình 2.2: Ngừng tìm kiếm sớm
(trên cây trò chơi)

- Thời gian (nhANH)
- Chính xác (phản ánh cơ hội chiến thắng trong thực tế)

2. TRÒ CHƠI CỜ VUA

- Ngừng tìm kiếm sớm
 - Giải thuật
 - Thay thế hàm tiện ích (**Utility**) bằng hàm đánh giá (**Eval**)

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if TO-MOVE}(s) = \text{MIN} \end{cases}$$

$$\text{H-MINIMAX}(s, d) = \begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if IS-CUTOFF}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if TO-MOVE}(s) = \text{MIN}. \end{cases}$$

2. TRÒ CHƠI CỜ VUA



- Hàm đánh giá

```
ScoreMap = { 'K':50, 'Q':9, 'R':5, 'N':3, 'B':3, 'P':1 }
```

```
def scorePie(board):  
    score = 0  
    for row in range(Size):  
        for col in range(Size):  
            piece = board[row][col]  
            if piece != None:  
                scoreP = 1 if piece[0]=='W' else -1  
                score += ScoreMap.get(piece[-1]) * scoreP  
  
    return score
```

2. TRÒ CHƠI CỜ VUA

- Hàm đánh giá

```
WhitePawnPos = [  
    [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],  
    [ 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0, 5.0],  
    [ 1.0, 1.0, 2.0, 3.0, 3.0, 2.0, 1.0, 1.0],  
    [ 0.5, 0.5, 1.0, 2.5, 2.5, 1.0, 0.5, 0.5],  
    [ 0.0, 0.0, 0.0, 2.0, 2.0, 0.0, 0.0, 0.0],  
    [ 0.5, -0.5, -1.0, 0.0, 0.0, -1.0, -0.5, 0.5],  
    [ 0.5, 1.0, 1.0, -2.0, -2.0, 1.0, 1.0, 0.5],  
    [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]]
```


2. TRÒ CHƠI CỜ VUA

- Hàm đánh giá

```
WhiteRockPos = [  
    [ 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0],  
    [ 0.5, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 0.5],  
    [-0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.5],  
    [-0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.5],  
    [-0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.5],  
    [-0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.5],  
    [-0.5, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.5],  
    [ 0.0, 0.0, 0.0, 0.5, 0.5, 0.0, 0.0, 0.0]]
```

2. TRÒ CHƠI CỜ VUA

- Xây dựng và cấu hình thuật giải Minimax-Alpha-Beta

```
def minValueAlphaBeta(board, depth, alpha, beta):  
    if board.is_checkmate():  
        score = - MAXScore if board.turn==False else MAXScore  
        return (score, None)  
    if depth == 0:  
        return (scorePiePos(board), None)  
  
    valMin = MAXScore  
    for move in board.legal_moves:  
        newBoard = board.copy()  
        newBoard.push(move)  
  
        val = maxValueAlphaBeta(newBoard, depth-1, alpha, beta)[0]  
        ...
```

1. Bài học đã cung cấp cho người học ví dụ về xây dựng trò chơi **Tic-tac-toe** với Minimax và Alpha-beta (sử dụng chiến lược **tìm kiếm tối ưu**).
2. Xây dựng trò chơi **Cờ vua** với Minimax và Alpha-beta (sử dụng **hàm ước lượng** để giảm độ phức tạp tính toán).

NHẬP MÔN TRÍ TUỆ NHÂN TẠO

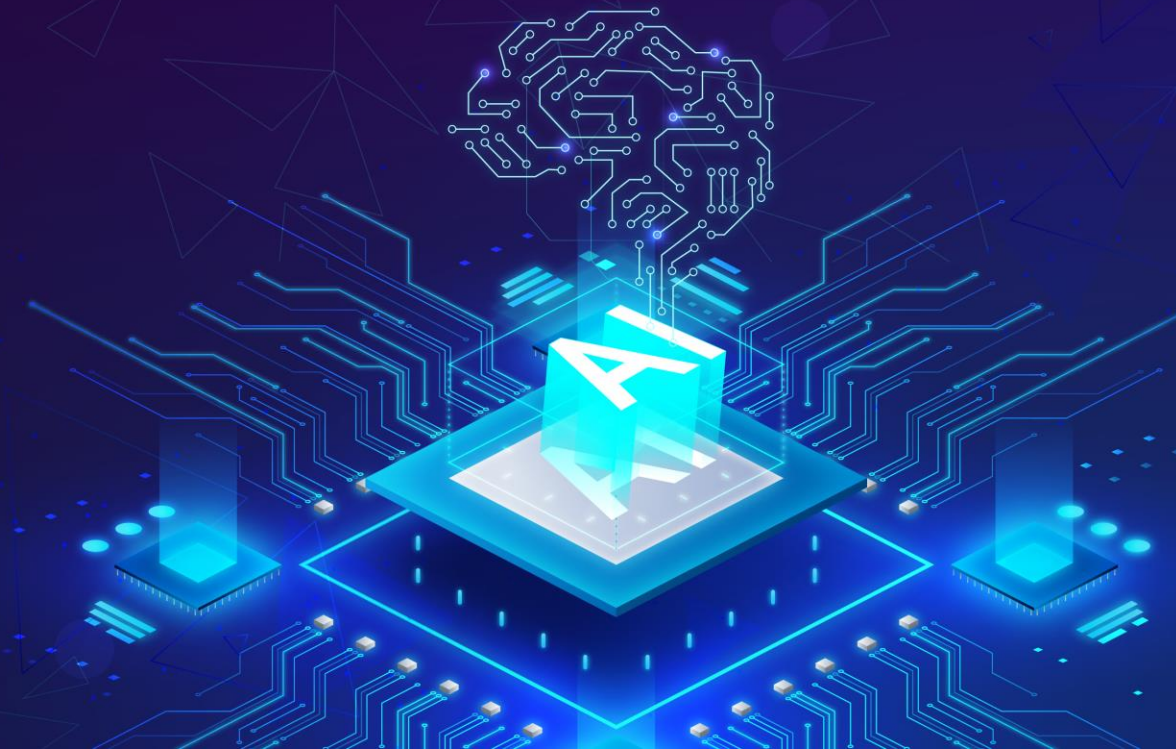
Ví dụ về tìm kiếm có đối thủ

Biên soạn:

TS. Đỗ Tiến Dũng

Trình bày:

TS. Đỗ Tiến Dũng



NHẬP MÔN TRÍ TUỆ NHÂN TẠO

Bài học tiếp theo:

Giới thiệu bài toán tìm kiếm thỏa mãn ràng buộc

Tài liệu tham khảo:

- [1] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach* (4th Edition). Prentice Hall, 2020
- [2] Từ Minh Phương. *Giáo trình nhập môn trí tuệ nhân tạo*. NXB Thông tin và truyền thông, 2016
- [3] T. M. Mitchell. *Machine Learning*. McGraw-Hill, 1997